

Proofbound: Compiling Heterogeneous Evidence into Verifiable Software Assurance

bordumb · bordumbb@gmail.com

1 September 2026

ABSTRACT

Software teams routinely present weak evidence in strong language: tests are quoted as guarantees, theorems about models are quoted as properties of shipping code, bounded checks are quoted without their bounds, and entire portfolios of claims are compressed into a single reassuring score. We present **Proofbound**, an assurance compiler that makes this laundering mechanically impossible. Registered claims are bound to exact subjects, evidence, and assumptions; a closed derivation compiles them into a three-facet status — formal standing, subject linkage, and assumption burden — that no adapter, checker, or operator can assert, upgrade, or average. The compiler emits content-addressed receipts, and a deliberately independent verifier that shares no code with the producer recomputes every status from the receipts alone; the two derivations are cross-checked against a language-neutral corpus of synthetic assurance graphs that includes deliberate status-upgrade attacks.

Proofbound integrates evidence that has historically lived in separate worlds: Lean 4 theorems audited from the elaborated environment, artifact bindings derived from the exact canonical encoding of a theorem’s statement rather than from checker output, Rust source refinement through qualified Charon/Aeneas translation driven entirely by manifests, Kani bounded checking with exact per-harness bounds, byte-reproducible distribution artifacts, static type checking, exact test and property-test inventories, and sealed single-mutant witnesses — across Rust, Python, and TypeScript subjects. Every route is a typed, fail-closed adapter with an authoritative inventory; a zero exit status is never evidence.

Adoption is tiered: a claim ledger over existing tests is useful on its own and was reached in under six minutes on an unmodified open-source crate, while the same manifests carry a claim to bounded checking, theorem proof, and mechanized refinement without restating its identity. Four pre-registered field studies on third-party codebases measured the system honestly — including a study that discovered, and forced the repair of, a soundness hole in the system’s own artifact binding. We argue that as machine-generated code outruns human review, the assurance receipt — not the reassuring dashboard — is the natural exchange format for trust between software producers and consumers.

1. Introduction

Every consequential software system makes claims: this decoder rejects malformed input; accepted transfers conserve value; this library implements the specification it names. What varies is not whether the claims exist but whether anyone can say, precisely, what supports them. In practice the answer is scattered across CI logs, test names, audit prose, and institutional memory — and the language used to report it is almost always stronger than the evidence underneath. A passing test suite becomes “correct.” A theorem about a hand-written model becomes a property of the code that ships. A model checker’s bounded result loses its bound in the retelling. A portfolio of heterogeneous obligations becomes “87% verified,” a number that erases exactly the distinctions it summarizes.

Every important claim is proved, explicitly assumed, tested, or visibly open — and no mechanism in the system can present one of these as another.

PROOFBOUND is a framework and toolchain built around that single invariant. It does not verify programs; decades of work on verified compilers and kernels do that far better than any orchestrator could (Leroy 2009; Klein et al. 2009). It solves the problem that sits above verification and below management: **compiling heterogeneous evidence into statuses that cannot overstate it, and making those statuses independently checkable by someone who does not trust the producer.**

Three observations motivate the design.

Evidence kinds are not interchangeable, but reporting flattens them. A kernel-checked theorem, a bounded model-checking run, a property-based test campaign, and an explicit assumption have different epistemic weight, different failure modes, and different trusted computing bases. Existing tooling reports each in its own vocabulary and leaves the composition to prose. Composition in prose is where the laundering happens: the well-studied gap between a verified model and a shipping implementation (Fonseca et al. 2017), the discovered non-independence of “independent” implementations (Knight and Leveson 1986), and the bound quietly dropped from a bounded claim are all failures of *composition reporting*, not of the underlying tools.

The producer of a status is the wrong party to certify it. A build orchestrator that both runs tools and computes verdicts can — through a bug or a shortcut — report success it did not earn. Proof-carrying code resolved the analogous problem by making the consumer’s checker small and independent of the producer (Necula 1997); certifying algorithms resolve it by emitting witnesses a simple checker can validate (McConnell et al. 2011). PROOFBOUND applies the same discipline to assurance itself: statuses travel as content-addressed receipts, and an independent verifier that shares no source with the orchestrator recomputes them from the receipts alone.

Machine-generated code makes both problems urgent. Large language models now produce a substantial fraction of new code, with measured security consequences (Pearce et al. 2022; Perry et al. 2023), and are increasingly capable of producing formal specifications and proofs as well (Misu et al. 2024). Generation volume breaks review-based trust at the same moment that generation collapses the historical cost objection to proof-carrying development. What survives that transition is a contract both sides can check mechanically: a registered claim with a fail-closed derivation is a precise, non-gameable objective for a producing agent, and a receipt is a precise, non-gameable deliverable for a consuming reviewer.

1.1 Contributions

1. **A three-facet status algebra** (§2) that decomposes a claim’s standing into formal strength, subject linkage, and assumption burden, derived — never asserted — from a validated evidence set under a named trust profile, with a closed evidence taxonomy that keeps theorem, refinement, bounded, transcribed, empirical, and assumed evidence permanently distinct, and a categorical refusal to emit aggregate scores.
2. **A typed assurance graph** (§2.5) whose legal edge table is encoded in the host language’s type system, revalidated at every deserialization boundary, and enforced identically by two independent implementations.
3. **Fail-closed evidence production** (§3): every external tool is wrapped in a typed adapter speaking a closed, canonical-JSON subprocess protocol from a sealed shadow of the reviewed tree, with authoritative inventories derived from tool metadata; artifact bindings are derived from the canonical encoding of the elaborated theorem statement itself, so no checker-authored boolean can create linkage; translation, transcription, mutation, and distribution routes each reproduce their results in independent executions before any status is granted.
4. **Independent receipt verification** (§4): a verifier with zero shared code recomputes graphs, digests, statement identities, and all three facets from receipts, and both derivations are cross-checked against a language-neutral adversarial corpus whose cases include producer-asserted status upgrades, evidence smuggling, and premise omission.
5. **A tiered adoption model** (§5) in which a claim ledger over existing tests is a complete, honest deliverable requiring no proof toolchain, and claim identities are stable as evidence strengthens — evaluated through demonstration verticals, self-application, and four pre-registered field studies on unmodified third-party code (§6), one of which broke and thereby improved the system’s own trust boundary.

1.2 Scope and terminology

A **claim** is a registered, stable-identity statement about a **subject** (source symbols, artifacts, or both) with cited evidence, assumptions, exclusions, and a **trust profile** naming what the claim’s policy admits. **Evidence units** are executable manifests that produce **receipts** through **adapters**. The **assurance graph** relates claims, theorems, subjects, artifacts, tests, assumptions, premises, policies, and trusted components with typed edges. **Compilation** validates the graph and derives statuses; **release** seals receipts for third parties; the **verifier** rechecks a release with no tool execution. Throughout, *fail closed* means that a malformed manifest, missing inventory, drifted artifact, or unknown field is an error that blocks status, never a warning.

This paper describes the system as specified and built; §7 states precisely what its outputs do and do not establish. It does not claim that any orchestrated tool — Lean, Kani, Charon, Aeneas, a compiler, a test runner — is itself verified, and it does not claim to bind object code.

2. The claim model and status algebra

2.1 Claims and claim closure

The unit of work in proof-driven development is not a function or a test but a **claim closure**: a stable identifier; exact internal and reader-facing statements; a bound subject; an evidence path; a transitive dependency graph; a transitive assumption set; a trust profile; a source and toolchain closure; a policy verdict; and a reproducible receipt. A theorem without a production subject is a model theorem, not a shipping claim. A checked artifact without a meaning theorem is checked data, not a proved claim. Registration is explicit: claims live in strict, schema-validated manifests whose unknown fields are rejected, and whose formal-declaration identity is an all-or-none triple of declaration name, canonical statement encoding, and statement digest (§3.2).

Source identity is class-separated. *Semantic* bytes can change a claim’s meaning; *runner* bytes can change how evidence is produced; *presentation* bytes touch neither; *external evidence* is content-addressed material outside production semantics. A claim binds only the closures relevant to its meaning, so a documentation edit does not invalidate a week of model checking, while a one-byte change to a semantic input invalidates exactly the receipts that depended on it.

2.2 Three orthogonal facets

A claim’s status is the composition of three facets:

$$\text{status}(c) = (F(c), L(c), A(c))$$

- **Formal facet** *F*: the strongest *policy-admitted* formal standing — PROVED, BOUNDED_CHECKED, TESTED, OPEN, or INVALID.
- **Linkage facet** *L*: how the formal object connects to the shipping subject — REFINED (a named refinement theorem over qualified-translated production code), ARTIFACT_BOUND (a digest-binding theorem over exact published bytes), TRANSCRIBED (a trusted external round trip), or MODEL_ONLY.
- **Assumption facet** *A*: NONE, or ASSUMED with the enumerated set of project axioms, representation premises, and external premises.

The facets are orthogonal by construction and never collapse into a scalar. PROVED·REFINED·ASSUMED is a common, honest terminal state: the theorem is real, the linkage is mechanized, and the representation premises are visible. Precedence takes the strongest admitted evidence for *F* while retaining weaker evidence in the record — displayed beneath the summary, never double-counted, never discarded. INVALID — any cited record missing, failed, drifted, unregistered, or ambiguous — overrides all facets and is simultaneously a reportable status and a build failure.

Two rules keep display language honest. A bounded claim’s public statement is derived: the claim’s own language followed by the literal registered finite-domain language, so no unbounded sentence is ever emitted for bounded evidence. And every human-facing report carries a mandatory “**not proved / out of scope**” section enumerating open obligations, undischarged premises, explicit assumptions, and registered exclusions; a report without it is invalid output, a rule enforced in the report deserializer itself.

Derivation is the core algorithm of the system and is closed: no adapter, plugin, or display layer may substitute its own mapping, and the status type has no writable field by which a caller could assert an outcome. What the producer derives, the independent verifier re-derives (§4.2); disagreement is a build failure.

2.3 Evidence taxonomy

Evidence kinds are a closed vocabulary; a claim may cite several kinds simultaneously, and the graph never conflates them.

Kind	Meaning (abbreviated)
theorem	A proof-assistant kernel accepted the named theorem.
artifact-soundness	A checked artifact identity equals the identity derived from an admitted theorem’s exact elaborated binding proposition.
trusted-transcription	Typed values transcribed outside the theorem boundary; byte identity enforced by an external round trip (§3.5).

Kind	Meaning (abbreviated)
source-refinement	Translated production code refines formal semantics under stated representation premises.
bounded-check	A model checker established the property over the registered finite domain.
independent-check	A deliberately independent implementation agreed on registered vectors.
exhaustive-check	Every member of a registered finite domain was evaluated.
property-test	Generated examples exercised a property; empirical.
example-test	Named test cases passed; empirical.
mutation-witness	A registered mutation of the subject violated a registered check (§3.7).
static-check	A registered analyzer reported zero violations over an exact inventory under a byte-pinned configuration; empirical.
review	A human attestation over a precisely scoped surface.
assumption / open	An explicit hypothesis; or absent required evidence.

Two qualifiers keep the strong kinds honest. Every theorem and artifact-soundness record carries an **evaluation mode** — kernel (ordinary elaboration) or native (compiled evaluation) — because native evaluation enlarges the trusted computing base and must be visible at the evidence level, not only in a profile. Every binding record carries a **binding mode**, and the modes are not interchangeable: a transcription round trip is never artifact soundness, and the taxonomy names the weaker shape rather than pretending it is the strong one.

Independence is treated as a claim requiring evidence, not a default. Where both sides of an “agreement” derive from one semantic source, the evidence is marked common-origin and cannot be presented as independent corroboration — a rule motivated directly by the empirical failure of assumed independence in multiversion programming (Knight and Leveson 1986; Avizienis 1985).

2.4 Assumptions, premises, and policy-gated discharge

Assumptions are first-class artifacts with stable identity, category, owner, rationale, scope, affected claims, review evidence, and a falsification or discharge plan. Categories separate mathematical hypotheses, representation premises, translator and compiler TCBs, runtime environments, external providers, cryptographic libraries, human attestations, and native evaluation.

Representation premises deserve emphasis because they are structural: source translation erases validated constructors and borrowed views into raw carriers, so every realistic refinement theorem takes validity structures as explicit hypotheses. Each such hypothesis is a premise node attached to its theorem, **undischarged by default**, and counted in the assumption facet unless a discharged-by edge connects it to another policy-admitted theorem proving the premise for the claim’s registered inputs. The absence of an edge can only weaken a status; forgetting one can never strengthen it. Discharge is never silent — a discharged premise remains visible with its discharging theorem — and it is policy-gated precisely so it cannot become a status-upgrade backdoor. A ledger label reading “discharged” is not discharge evidence; the derivation rejects it.

2.5 The typed assurance graph

The compiled graph has fourteen node kinds and thirteen edge kinds whose legal endpoint pairs form a closed table of twenty-two entries (proves connects theorems to claims and nothing else; checks connects test suites and model-check units to claims; inventory nodes such as toolchains have no legal edge at all and may not acquire an invented relationship merely to be connected). The table is encoded three times from one source: as a compile-time constraint — edge constructors accept only marker-typed endpoint references, a generic unchecked edge constructor is not public, and a compile-fail test demonstrates that a kind-correct but relation-invalid edge is rejected by the type checker — as a runtime predicate revalidated on every deserialization, because canonical JSON is an untrusted input that can necessarily represent an illegal edge; and independently in the verifier (§4.2). Cycles are legal only for declared mutual theorem dependencies inside one proof environment; cycles in generation or provenance are invalid.

2.6 Trust profiles

Policies are named profiles, not conventions. The built-ins — `ledger`, `transcribed`, `kernel`, `kernel-with-assumptions`, `artifact-bound`, `source-refined`, `native-evaluated`, and `bounded` — form a closed set whose meanings projects may strengthen but never redefine. `ledger` is the Tier 0 profile: it admits empirical and assumed evidence only, its strongest formal facet is `TESTED`, and stronger evidence may remain visible but cannot promote a `ledger` claim. `kernel` admits no project axioms and no native evaluation; `artifact-bound` requires the exact typed binding of §3.3; `bounded` requires the receipt's solver and per-harness unwind bounds to equal the registered model-check unit exactly. Profile admission is checked per theorem: a rejected strong record stays in the output with its rejection reasons, because hiding rejected evidence is itself a form of laundering.

3. Producing evidence without trusting producers

3.1 The adapter boundary

Adapters turn external tool results into canonical evidence, and the boundary is designed so that an adapter *cannot* fabricate assurance even if it wants to.

Adapters are separate processes speaking a versioned JSON protocol over `stdin/stdout`. Requests and responses are schema-validated canonical JSON; a response must be byte-identical to its own canonical re-encoding, must echo a request identity derived by hashing the request, and must not carry evidence on failure. An adapter returns either a complete evidence record or — the common case — a strict **execution observation**: the complete ordered command array, an equally sized run array binding each command's exit state, raw and normalized output identities, truncation state, and duration. The assurance compiler, not the adapter, enriches observations with graph and closure identities, so a tool adapter cannot author project provenance it does not own. Evidence derived wholly inside the compiler is marked `compiler-internal` with empty command arrays; inventing a process record for an internal derivation is invalid. Because the protocol is a wire contract rather than a linking convention, an adapter may be written in any language without touching the core.

Execution is sealed. Adapters run in shadows copied from the reviewed tree with cleared environments and explicit per-unit allowlists, symlinks rejected, budgets enforced, and typed argument construction throughout — no shell string ever reaches a tool. Inventories are authoritative and bidirectional: registered Lean claims are enumerated from compiled-environment attributes, Kani harnesses from tool metadata, test nodes from runner listings, and a configured target that is silently skipped — or a discovered target that is silently unregistered — fails the build. The system's verification command never writes to the reviewed tree; a single separate update command is the only door through which committed generated state changes, and it requires a clean tree and produces a reviewable diff.

3.2 Statement identity for elaborated theorems

Text is not identity. For every registered theorem, the Lean adapter compiles an audit from the elaborated environment — never from source scanning — recording the fully qualified declaration, its transitive axioms partitioned into foundational and project axioms, and a digest of the statement under a versioned canonical encoding (`lean-expr-cbor/1`): canonical CBOR over the elaborated expression with de Bruijn indices, no binder names, no positions, explicit universes and literals, and rejection of metavariables and free variables, hashed under a domain separator. A silently restated theorem therefore renders its claim `INVALID` rather than quietly proving something else. `sorry-axioms` and undeclared project axioms are rejected; attribute-based claim discovery is bidirectional, and a public-claim surface requires every declaration to be attributed or exempted with a recorded reason — an unattributed theorem can exist, but it cannot be published.

3.3 Pattern A: canonical artifact soundness

When untrusted or heuristic code emits a compact certificate that is cheaper to check than to discover — the certifying-algorithms shape (McConnell et al. 2011) — `PROOFBOUND` binds the certificate to its meaning through the theorem statement itself. An admitted binding theorem must have as its *exact elaborated root* the versioned typed proposition `DigestBindingV1 claim schema name digest bytes meaning` with the first four arguments direct string literals; the proposition establishes both the SHA-256 identity of the bytes and their domain meaning. The compiler parses the canonical statement wire, recovers the literal claim, name, and digest *from theorem content*, and joins them to the artifact identity recomputed by a separately registered checker — including byte size. The independent verifier repeats the parse and the join from the portable wire. A checker outcome, a theorem name, or a manifest boolean has no status-

bearing representation anywhere on this path: there is nothing an adapter can say that creates `ARTIFACT_BOUND`. Section 6.4 recounts how a field study demonstrated why this derivation, and not a checker-authored flag, must be the boundary.

3.4 Pattern B: translated source refinement

When a small pure production kernel is itself the subject, the kernel is translated to Lean by Charon and Aeneas (Ho and Protzenko 2022) and related by named theorems to rich handwritten semantics — the refinement shape of verified systems (Leroy 2009; Klein et al. 2009), applied at the scale of a project’s decision kernels rather than a compiler. PROOFBOUND treats the translation as untrusted output requiring qualification, in the spirit of translation validation (Pnueli, Siegel, and Singerman 1998):

- **Manifests drive invocation.** The translation unit declares typed, ordered tool invocations — exact package manifests, crate and LLBC identities, start/opaque/include selectors, and a closed produced-to-destination output map. No symbol list, path mapping, or unit count exists in orchestration source; adding a unit is a manifest change.
- **The translated closure is pre-registered and compared bidirectionally.** The complete supported local function and type closure Aeneas is expected to report is part of the manifest; an empty, missing, extra, duplicate, or unsupported entry is not evidence even when both tools exit zero.
- **Determinism is demonstrated, not presumed.** Every translation runs twice and must agree byte-for-byte, with the sole audited nondeterministic surface (raw LLBC) compared under a declared pretty-printed normalization.
- **Generated code is quarantined.** The generated directory is exclusively generator-owned; hand-written external bridges live outside it, byte-pinned and declared; translator placeholder axioms are quarantined uncompiled with exact counts; a new upstream warning fails rather than scrolls past.

Kani remains complementary bounded evidence on the same kernels — it sees Rust control flow and machine arithmetic where Lean sees the mathematical definition — and is never presented as the refinement bridge.

3.5 Named degradations: trusted transcription

A common weaker binding transcribes orchestrator-decoded values into typed Lean literals and enforces byte identity with an external re-encoder. PROOFBOUND admits this route but names it: a fixed two-command driver ABI transcribes the source and re-encodes the *fresh candidate* in one connected execution; the candidate must equal the committed transcription and the re-encoding must equal the source, byte-for-byte; the transcriber and re-encoder are derived as distinct trusted-computing-base roles even when one driver implements both. The result is `TRANSCRIBED` linkage — never `ARTIFACT_BOUND`, never `PROVED` — and the profile for the strong pattern rejects it. Naming the degradation, rather than inheriting the ambiguity of systems that ship both shapes under one label, is the taxonomy doing its job.

3.6 Bounded model checking

Bounded evidence is admitted only against a registered finite domain with explicit cardinality and ordering, checked by Kani (Delmas et al. 2026), the Rust front end in the CBMC lineage (Kroening, Schrammel, and Tautschnig 2023). Harness inventories come from tool metadata and are matched bidirectionally; the receipt’s solver must equal the registered solver; the unwind-bound key set must equal the harness set with every bound the registered nonzero value; and the registered execution-model assumptions are preserved verbatim and in order — never trimmed, classified, or substituted. No unbounded language is ever emitted for bounded evidence (§2.2).

3.7 Empirical routes

Empirical evidence is held to the same inventory discipline as formal evidence. Test routes discover nodes from runner metadata, execute each registered node individually, and require that exactly one test ran and passed per node — for Rust, Python, and TypeScript subjects alike, with package-manager lifecycle scripts disabled unconditionally in the JavaScript ecosystem. Property-test runs register their framework and seed and are reported as observed campaigns, never as coverage of a search space (Claessen and Hughes 2000; MacIver and Hatfield-Dodds 2019). Static type checks are static-check evidence: zero violations over an exact inventory under a byte-pinned strict configuration, capped at `TESTED`.

Mutation witnesses follow the original insight that a test’s value is the faults it can reject (DeMillo, Lipton, and Sayward 1978; Jia and Harman 2011), in sealed singleton form: a registry pins one full-file mutant by digest and one exact witness test; the adapter verifies the target preimage, runs the witness in a clean shadow (it must pass), installs the

mutant in an independent shadow, verifies the postimage, and reruns the witness (it must fail with the runner’s exact tests-failed exit status). Aggregate mutation scores are not evidence — they are the scalar the system exists to refuse — but each registered witness is a reproducible demonstration that a named guard is load-bearing.

Distribution reproduction binds the bytes users install to the reviewed tree: wheels, sdists, and npm packages are built twice in independent shadows and must agree byte-for-byte and equal a registered digest, with member inventories verified against the archive’s own manifest — reproducible-builds discipline (Lamb and Zacchiroli 2022) applied per claim rather than per distribution.

4. Independent verification of receipts

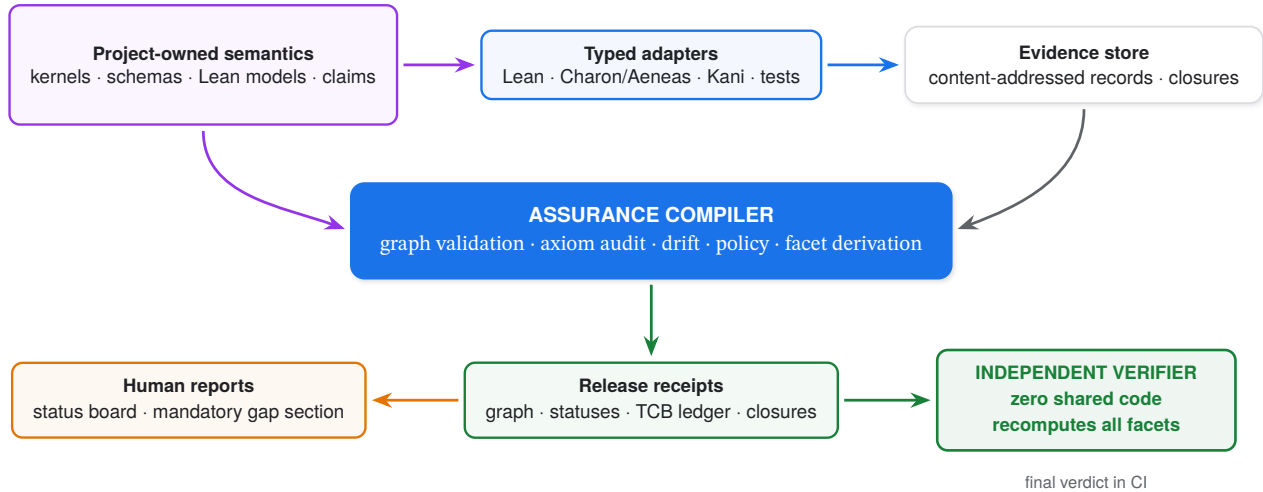


Figure 1: The evidence pipeline. Adapters observe tool executions from sealed shadows; the compiler derives facets; receipts are the only interface to reporting and to the independent verifier, whose verdict — not the producer’s — is the verdict CI reports.

4.1 Receipts and provenance

Every evidence record binds project revision and tree state, semantic closure, exact input and generated artifact digests, complete tool identity, every typed command in execution order with its aligned run record, a separate typed reproduction command, timing, resource budget and observed usage (an unmeasured peak is an explicit null, never an invented zero), and adapter version. Receipts are canonical and content-addressed; human reports are projections of receipts and never sources of truth. Releases add package checksums, canonical schemas, the compiled graph, per-claim receipts, the assumption ledger, a strict trusted-computing-base ledger whose component set must equal the union recomputed from evidence provenance, and source and toolchain closures.

4.2 The independent verifier

The orchestrator both runs tools and computes statuses; a bug there could falsely report success. The verifier is therefore a separate minimal program — not a subcommand — that shares no code with the orchestrator, executes no external tools, and reads only a release. It revalidates schemas, digests, closure membership, path safety, and the complete edge table; re-encodes the canonical statement wire and rechecks every theorem’s statement identity; recognizes only the exact typed binding root of §3.3; re-derives all three facets for every claim under §2’s rules; and rejects any reported status stronger than its own recomputation. The duplication is deliberate and is held to the framework’s own standard for independent checks: both implementations derive from the same specification, share no source, and are cross-checked continuously (§4.3).

Its trust boundary is stated, not implied: the verifier certifies that reported statuses are **receipt-consistent** — that graph and facets follow from the recorded evidence. It cannot attest that Lean, Kani, or any tool actually ran honestly; that remains bound by the receipts’ tool identities, closures, and reproduction commands, and its output language is capped accordingly.

4.3 The adversarial conformance corpus

Both status engines are cross-checked against a registered corpus of language-neutral synthetic assurance graphs — deliberately not a serialization of any implementation type — covering every formal and linkage facet and, critically, **attack cases**: a producer-asserted status upgrade, omitted assumptions, omitted premises, empirical evidence presented as proof, an unrelated theorem smuggling a nested binding marker, transcription evidence offered against an artifact-bound profile, and forged artifact sizes. For each attack the corpus fixes the required outcome: the producer must refuse to derive the stronger status, and the verifier must independently reject a release asserting it. Divergence between the two engines on any case fails CI. The corpus grows monotonically; every field-discovered laundering shape (§6.4) becomes a permanent case.

4.4 Caching changes cost, never meaning

Fail-closed re-verification of everything on every invocation would not be run by humans, and a verification tool nobody runs verifies nothing. Receipts are therefore reused when a cache key — semantic closure digest, input digests, toolchain and adapter identity, and the complete unit configuration — is unchanged, with reuse recorded in the receipt chain and reported distinctly from fresh verification. A cached receipt asserts exactly what the original run asserted against exactly the same closure; an unverifiable cached receipt causes re-execution, never acceptance.

4.5 Assurance regression control

Between any two revisions, the system classifies **assurance regressions**: a new assumption or newly undischarged premise, an enlarged TCB, a linkage or evaluation-mode downgrade, a formal-facet downgrade, a narrowed or incomparable bounded domain, removed mutation coverage, or a weakened closure. Continuous integration runs verify-only gates for every event class — no CI event may regenerate artifacts and accept the result — and rejects any change carrying a regression unless it includes an approval: a first-class review record bound to the exact base and head revision digests, enumerating the specific regressions it approves, itself graph evidence that is invalidated if the diff it approved changes. Approvals are add-only and bound to reviewed subjects, so the gate cannot be satisfied circularly. A regression without an approval fails; an approval without a matching regression is rejected as stale.

5. The adoption ladder

Full proof-driven development is expert work, and no orchestrator changes that. What PROOFBOUND changes is how much of the value is available before the expert work begins — a gradual-verification posture (Bader, Aldrich, and Tanter 2018) applied to evidence rather than to type systems:

Tier	Adds	Requires	Strongest status
0	Claim ledger: claims, explicit assumptions, existing tests bound as evidence	the CLI only	TESTED / ASSUMED / OPEN
1	Trusted transcription; bounded checking	a registered driver and/or Kani	BOUNDED_CHECKED, TRANSCRIBED
2	Model theorems; compiled axiom audit	Lean toolchain	PROVED · MODEL_ONLY
3	Source refinement; artifact binding	Charon/Aeneas; digest theorems	PROVED · REFINED / ARTIFACT_BOUND

Tier 0 is deliberately valuable alone: most teams have never enumerated their load-bearing claims, and an honest board of TESTED/ASSUMED/OPEN over an existing suite is a real deliverable. Brownfield adoption inverts the green-field order — inventory the claims the system already silently makes, bind the tests it already runs, register what the team already knows it is trusting (the step that reliably surfaces the first surprises), and promote individual claims only where value justifies proof-engineering cost. Tiers are a per-project floor and per-claim ceiling; promotion never restates a claim's identity. The same ladder governs language ecosystems: any conventional repository reaches the governance level; Rust, Python, and TypeScript reach exact empirical evidence; Rust additionally reaches bounded

checking and mechanized refinement — and the capability level is reported honestly rather than compressed into a “supported” badge.

6. Evaluation

Four questions matter. Can a stranger’s codebase reach an honest board quickly? Do the demonstrations exercise the full algebra, including its refusals? Does the system apply its own discipline to itself? And when pressed adversarially, does it launder — or break loudly and improve?

6.1 Demonstration verticals

The **allowance** vertical registers seven claims over a transfer decision pipeline: conservation, no-overdraft, cap-respect, and denial-immutability laws proved in Lean over a model refined from the pure Rust kernel; canonical request bytes decoded independently in Rust and Lean; four Kani harnesses covering the registered 2^{34} -state request domain; five registered mutation witnesses; and — deliberately — one claim that can only ever be TESTED (the orchestrator submits the exact receipted bytes) and one explicit external assumption (the identity provider’s authorization response is truthful) that Lean does not pretend to prove. The demo classifying its own weakest claims honestly is part of the demonstration.

The **artifact-certificate** vertical exercises Pattern A end-to-end: independently written Lean, Rust, and Python implementations of a canonical certificate format; digest-binding theorems whose literal bytes equal the published fixtures; one axiom-free claim and one claim carrying a deliberately registered domain axiom with its own profile — so the difference between “proved” and “proved under a named hypothesis” is visible in a working system. The **transcription** vertical demonstrates the degraded binding of §3.5 deriving exactly OPEN · TRANSCRIBED — a status whose honesty is the feature.

6.2 Self-application

PROOFBOUND registers claims about itself: manifests reject unknown fields; status cannot be manually upgraded; undeclared project axioms invalidate theorem claims; ungated harnesses invalidate bounded coverage; stale generated translation invalidates refinement; receipt identity tracks semantic source; presentation-only changes leave semantic closures untouched; the two status engines agree over the conformance corpus; and the CI gates themselves are registered subjects, so a stale gate surfaces as INVALID rather than as silence. Self-application does not pretend the orchestrator is formally verified — its own board shows which of its properties are proved, tested, or open. The repository gate runs the twelve normative stages — cheap deterministic checks first, one fresh assurance compilation, demo verification, release reproduction — and ends with the independent verifier as the final verdict; a full gate concludes with every registered claim receipt-consistent and publication admitted.

6.3 Pre-registered field studies

Four field studies were run against unmodified third-party codebases, each pre-registered — falsifiable questions with pass criteria committed before the subject work began — with append-only journals and pinned subjects (exact revisions or content-addressed crate archives). Two subjects were the mature verification projects whose patterns PROOFBOUND generalizes; two were ordinary widely used open-source crates (semver 1.0.28 and base64 0.22.1).

Measured results: an honest Tier 0 board on semver in **5 minutes 25 seconds** from initialization, deriving TESTED · MODEL_ONLY · ASSUMED; a corrected board on base64 in under 26 minutes; four Kani harnesses over base64’s decode paths covering 140{,}290 registered cases at the registered bounds; and a seeded fault whose only trace was a claim flipping to INVALID and blocking publication — the fail-closed path demonstrated on foreign code. Extracting a pure decision kernel from semver for Pattern B cost +180/ – 4 production lines, a measured datum for the real price of refinement-readiness.

The studies’ outcome vocabulary permitted **fail** and **unanswered**, and both were used: of nineteen pre-registered questions, eight passed, seven failed, and four were recorded as unanswered rather than stretched. Every divergence between expectation and reality — 28 in total — received a written disposition: a specification change, an accepted limitation, or a repair. No aggregate score of the studies is reported anywhere, including here.

6.4 What the field studies broke

The most valuable result was a failure. In the first Pattern A run on base64, checker-authored binding booleans allowed ARTIFACT_BOUND linkage even though the audited theorem never mentioned the artifact’s digest — precisely the

laundering the system exists to prevent, inside the system itself. The repair removed checker-authored linkage from the trust boundary entirely: binding is now derived only from the exact elaborated root proposition (§3.3), recomputed independently by producer and verifier, and the attack — an unrelated theorem smuggling a nested binding marker — is a permanent corpus case that both engines must reject. Previously accepted receipts that predated the stricter rules were **revoked, not grandfathered**, on two occasions: once for bounded receipts that had discarded registered solver and unwind identities, and once for the binding change. A system whose thesis is that evidence must never be presented as stronger than it is must apply that rule to its own past output; the revocations are the demonstration.

7. Assurance boundary

This paper does not claim that Proofbound verifies programs, nor that a compiled status is a proof of fitness.

A status is a mechanically derived, independently recheckable summary of registered evidence under a named policy. Its honesty is the contribution; its strength is exactly the strength of the underlying evidence.

What a compiled status establishes. That the cited evidence records exist, validate, and bind the exact subjects, closures, and toolchains named; that the facets follow from the closed derivation under the claim’s profile; that every assumption and undischarged premise in the closure is enumerated in the output; and that an independent implementation reaches the same verdict from the receipts alone.

What is trusted. The proof assistant’s kernel and the compiled audit executable; the model checker, translator, compilers, and runtimes named in the TCB ledger; native evaluation where a profile admits it, surfaced per record; the transcriber and re-encoder roles of §3.5; the operating system and hardware; and, for each claim, the explicitly registered external premises. The ledger is exact: a component the receipts did not record cannot appear in it, and a missing or conflicting component invalidates the release.

What cannot be attested. That an orchestrated tool executed honestly rather than being substituted — reproduction commands and tool identities bound in receipts make dishonesty detectable on re-run, not impossible; that a formal statement means what its prose claim says — statement identity closes the drift channel, but semantic adequacy of the formalization remains a human review obligation, the enduring core of the classic skepticism about program proofs (DeMillo, Lipton, and Perlis 1979); and that a model is the right model — a correct proof of the wrong property is still wrong, which is why every report carries its “not proved / out of scope” section rather than a conclusion.

8. Related work

8.1 Proof-carrying artifacts and certifying computation

Proof-carrying code made the producer supply evidence a small consumer checker validates (Necula 1997); certifying algorithms made programs emit per-run witnesses (McConnell et al. 2011); Hoare’s verifying compiler posed the grand challenge of mechanized correctness as a routine artifact (Hoare 2003). PROOFBOUND carries the producer/checker asymmetry one level up: the artifact that travels is not a proof of a program but a receipt set over heterogeneous evidence, and the small trusted consumer is the independent verifier. Unlike PCC, PROOFBOUND does not require the strong evidence to exist — it requires the *classification* to be sound whatever the evidence is.

8.2 Verified systems and translation validation

CompCert and seL4 demonstrated end-to-end refinement with explicit trusted bases (Leroy 2009; Klein et al. 2009), and empirical study of verified systems shows failures concentrate precisely in the unverified gaps between proof and world (Fonseca et al. 2017). PROOFBOUND is designed for that finding: it does not extend proofs into the gaps; it forces the gaps to be registered objects. Its translation qualification is a translation-validation posture (Pnueli, Siegel, and Singerman 1998) — per-result checking of an untrusted translator — rather than translator verification.

8.3 Assurance cases

Structured assurance arguments — GSN (Kelly and Weaver 2004), their formal interpretation (Rushby 2015), and Assurance 2.0’s push toward rigorous, defeater-aware cases (Bloomfield and Rushby 2021) — share PROOFBOUND’s premise that assurance is an explicit argument over evidence with visible assumptions. The difference is execution: an assurance case is authored and reviewed; a PROOFBOUND graph is compiled, fails closed, re-derives on every change,

and is recomputed by an independent implementation. Defeaters have an executable analogue in the attack corpus and mutation witnesses; eliminative argumentation has one in the mandatory not-proved section.

8.4 Supply-chain integrity and transparency

in-toto attests who performed which supply-chain step (Torres-Arias et al. 2019); Sigstore makes signing and identity binding practical (Newman, Meyers, and Torres-Arias 2022); SLSA levels build integrity (Open Source Security Foundation 2023); reproducible builds tie binaries to sources (Lamb and Zacchiroli 2022); Certificate Transparency made a trust ecosystem auditable by log (Laurie, Langley, and Kasper 2013). These systems answer *who built what, from which bytes*. PROOFBOUND answers the question above it: *what is known about what was built, and on what evidence*. The layers compose: a PROOFBOUND release is itself an artifact whose provenance those systems can carry, while its receipts carry the semantics they cannot.

8.5 Testing, independence, and mutation

QuickCheck and its descendants made property-based evidence cheap (Claessen and Hughes 2000; MacIver and Hatfield-Dodds 2019); differential testing finds disagreement between implementations (McKeeman 1998); N-version programming sought reliability through independent implementations (Avizienis 1985) and Knight and Leveson showed the independence assumption empirically false (Knight and Leveson 1986); mutation analysis grounds test adequacy in fault rejection (DeMillo, Lipton, and Sayward 1978; Jia and Harman 2011). PROOFBOUND encodes these lessons as vocabulary: empirical kinds that cannot promote to proof, common-origin evidence that cannot claim independence, and registered singleton mutation witnesses in place of aggregate scores.

8.6 Rust verification

RustBelt secured the foundations (Jung et al. 2018); Verus builds SMT-backed verification into Rust development (Lattuada et al. 2024); Kani provides bit-precise bounded checking (Delmas et al. 2026); Aeneas translates safe Rust to functional definitions amenable to interactive proof (Ho and Protzenko 2022). PROOFBOUND orchestrates rather than competes: Aeneas supplies the refinement route because its output can be related directly to handwritten Lean semantics (Moura and Ullrich 2021; The mathlib Community 2020), Kani supplies independent bounded evidence at the MIR level, and the framework’s contribution is that neither can be quoted as the other.

8.7 Machine-generated code

Measured studies show AI assistants produce insecure code at meaningful rates and induce overconfidence in their users (Pearce et al. 2022; Perry et al. 2023), while parallel work shows models increasingly able to produce verified artifacts when the target is a checkable specification (Misu et al. 2024). Both findings point the same direction: the reviewable unit for machine-generated software should be a machine-checkable claim, not a diff. A registered claim with a fail-closed derivation is an objective a generation loop cannot satisfy by plausible-looking output, and the adversarial corpus is, in effect, a hardening of the grader against the generator. Gradual adoption (Bader, Aldrich, and Tanter 2018) matters here for the same reason it matters for humans: the ladder from TESTED to PROVED · REFINED gives an agent — and its operator — somewhere honest to stand at every step.

9. Conclusion

PROOFBOUND exists to make the boundary of knowledge executable. It asks teams neither to choose between ordinary testing and total verification, nor to pretend the choice away with a score. State the claim; prove what can be proved; link the proof to what ships; test what remains empirical; name every assumption; refuse to hide the gaps; and hand the whole account to a verifier that does not trust you. The individual disciplines are decades old — proof-carrying evidence, refinement, translation validation, mutation adequacy, reproducible artifacts, transparency. The contribution is the compiler that holds them together and the receipt that lets a stranger check the result.

The timing argument is the vision. Software is entering a period in which most code will be written by systems that cannot be cross-examined and reviewed by people who cannot keep up. In that world, trust either degenerates into brand reputation or is rebuilt on artifacts. An assurance receipt — portable, facet-preserving, independently recheckable, honest about its own limits — is the artifact we propose. Make your software’s claims compile.

References

- Avizienis, Algirdas. 1985. “The N-Version Approach to Fault-Tolerant Software.” *IEEE Transactions on Software Engineering* SE-11 (12): 1491–501. <https://doi.org/10.1109/TSE.1985.231893>.
- Bader, Johannes, Jonathan Aldrich, and Éric Tanter. 2018. “Gradual Program Verification.” In *Verification, Model Checking, and Abstract Interpretation*, 10747:25–46. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/978-3-319-73721-8_2.
- Bloomfield, Robin, and John Rushby. 2021. “Assurance 2.0: A Manifesto.” *arXiv Preprint arXiv:2004.10474*. <https://doi.org/10.48550/arXiv.2004.10474>.
- Claessen, Koen, and John Hughes. 2000. “QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs.” In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, 268–79. <https://doi.org/10.1145/351240.351266>.
- Delmas, Rémi, Ziyad Hassan, Qinheping Hu, Rahul Kumar, Felipe R. Monteiro, Thanh Nguyen, Adrián Palacios, et al. 2026. “Kani: A Model Checker for Rust.” *arXiv Preprint arXiv:2607.01504*. <https://doi.org/10.48550/arXiv.2607.01504>.
- DeMillo, Richard A., Richard J. Lipton, and Alan J. Perlis. 1979. “Social Processes and Proofs of Theorems and Programs.” *Communications of the ACM* 22 (5): 271–80. <https://doi.org/10.1145/359104.359106>.
- DeMillo, Richard A., Richard J. Lipton, and Frederick G. Sayward. 1978. “Hints on Test Data Selection: Help for the Practicing Programmer.” *IEEE Computer* 11 (4): 34–41. <https://doi.org/10.1109/C-M.1978.218136>.
- Fonseca, Pedro, Kaiyuan Zhang, Xi Wang, and Arvind Krishnamurthy. 2017. “An Empirical Study on the Correctness of Formally Verified Distributed Systems.” In *Proceedings of the Twelfth European Conference on Computer Systems*, 328–43. <https://doi.org/10.1145/3064176.3064183>.
- Ho, Son, and Jonathan Protzenko. 2022. “Aeneas: Rust Verification by Functional Translation.” *Proceedings of the ACM on Programming Languages* 6 (ICFP): 711–41. <https://doi.org/10.1145/3547647>.
- Hoare, Tony. 2003. “The Verifying Compiler: A Grand Challenge for Computing Research.” *Journal of the ACM* 50 (1): 63–69. <https://doi.org/10.1145/602382.602403>.
- Jia, Yue, and Mark Harman. 2011. “An Analysis and Survey of the Development of Mutation Testing.” *IEEE Transactions on Software Engineering* 37 (5): 649–78. <https://doi.org/10.1109/TSE.2010.62>.
- Jung, Ralf, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. 2018. “RustBelt: Securing the Foundations of the Rust Programming Language.” *Proceedings of the ACM on Programming Languages* 2 (POPL): 66:1–34. <https://doi.org/10.1145/3158154>.
- Kelly, Tim, and Rob Weaver. 2004. “The Goal Structuring Notation – a Safety Argument Notation.” In *Proceedings of the DSN 2004 Workshop on Assurance Cases*. <https://www-users.york.ac.uk/~tpk1/dsn2004.pdf>.
- Klein, Gerwin, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, et al. 2009. “seL4: Formal Verification of an OS Kernel.” In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles*, 207–20. <https://doi.org/10.1145/1629575.1629596>.
- Knight, John C., and Nancy G. Leveson. 1986. “An Experimental Evaluation of the Assumption of Independence in Multiversion Programming.” *IEEE Transactions on Software Engineering* SE-12 (1): 96–109. <https://doi.org/10.1109/TSE.1986.6312924>.
- Kroening, Daniel, Peter Schrammel, and Michael Tautschnig. 2023. “CBMC: The C Bounded Model Checker.” *arXiv Preprint arXiv:2302.02384*. <https://doi.org/10.48550/arXiv.2302.02384>.
- Lamb, Chris, and Stefano Zacchiroli. 2022. “Reproducible Builds: Increasing the Integrity of Software Supply Chains.” *IEEE Software* 39 (2): 62–70. <https://doi.org/10.1109/MS.2021.3073045>.
- Lattuada, Andrea, Travis Hance, Jay Bosamiya, Matthias Brun, Chanhee Cho, Hayley LeBlanc, Pranav Srinivasan, et al. 2024. “Verus: A Practical Foundation for Systems Verification.” In *Proceedings of the 30th ACM Symposium on Operating Systems Principles*, 438–54. <https://doi.org/10.1145/3694715.3695952>.
- Laurie, Ben, Adam Langley, and Emilia Kasper. 2013. “Certificate Transparency.” RFC 6962. RFC Editor. <https://doi.org/10.17487/RFC6962>.
- Leroy, Xavier. 2009. “Formal Verification of a Realistic Compiler.” *Communications of the ACM* 52 (7): 107–15. <https://doi.org/10.1145/1538788.1538814>.
- MacIver, David R., and Zac Hatfield-Dodds. 2019. “Hypothesis: A New Approach to Property-Based Testing.” *Journal of Open Source Software* 4 (43): 1891. <https://doi.org/10.21105/joss.01891>.
- McConnell, Ross M., Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. 2011. “Certifying Algorithms.” *Computer Science Review* 5 (2): 119–61. <https://doi.org/10.1016/j.cosrev.2010.09.009>.
- McKeeman, William M. 1998. “Differential Testing for Software.” *Digital Technical Journal* 10 (1): 100–107. <https://www.cs.swarthmore.edu/~bylvisa1/cs97/f13/Papers/DifferentialTestingForSoftware.pdf>.
- Misu, Md Rakib Hossain, Cristina V. Lopes, Iris Ma, and James Noble. 2024. “Towards AI-Assisted Synthesis of Verified Dafny Methods.” *Proceedings of the ACM on Software Engineering* 1 (FSE). <https://doi.org/10.1145/3643763>.
- Moura, Leonardo de, and Sebastian Ullrich. 2021. “The Lean 4 Theorem Prover and Programming Language.” In *Automated Deduction – CADE 28*, 12699:625–35. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/978-3-030-79876-5_37.
- Necula, George C. 1997. “Proof-Carrying Code.” In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 106–19. <https://doi.org/10.1145/263699.263712>.
- Newman, Zachary, John Speed Meyers, and Santiago Torres-Arias. 2022. “Sigstore: Software Signing for Everybody.” In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2353–67. <https://doi.org/10.1145/3548606.3560596>.
- Open Source Security Foundation. 2023. “SLSA: Supply-Chain Levels for Software Artifacts, Version 1.0.” Specification. <https://slsa.dev/spec/v1.0/>.
- Pearce, Hammond, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. “Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions.” In *2022 IEEE Symposium on Security and Privacy*, 754–68. <https://doi.org/10.1109/SP46214.2022.9833571>.
- Perry, Neil, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. “Do Users Write More Insecure Code with AI Assistants?” In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2785–99. <https://doi.org/10.1145/3576915.3623157>.
- Pnueli, Amir, Michael Siegel, and Eli Singerman. 1998. “Translation Validation.” In *Tools and Algorithms for the Construction and Analysis of Systems*, 1384:151–66. Lecture Notes in Computer Science. Springer. <https://doi.org/10.1007/BFb0054170>.
- Rushby, John. 2015. “The Interpretation and Evaluation of Assurance Cases.” Technical Report SRI-CSL-15-01. Computer Science Laboratory, SRI International. <https://www.csl.sri.com/users/rushby/papers/sri-csl-15-1-assurance-cases.pdf>.
- The mathlib Community. 2020. “The Lean Mathematical Library.” In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, 367–81. <https://doi.org/10.1145/3372885.3373824>.
- Torres-Arias, Santiago, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Capps. 2019. “In-Toto: Providing Farm-to-Table Guarantees for Bits and Bytes.” In *28th USENIX Security Symposium*, 1393–1410. USENIX Association. <https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias>.